FRMHG • Architecture Globale • MVP + Extensible

SPORTIFY®

Plateforme de Gestion
Fédération Royale Marocaine de Hockey sur Glace

Document d'architecture : modules fonctionnels, topologie technique (1 serveur on-prem), sécurité, sauvegardes, flux critiques et modèl de données minimal.

Cible
50 clubs • ~2000 utilisateurs

Déploiement
Docker Compose • 1 serveur

Axes
Gouvernance • Sécurité • Audit

Monolithe modulaire
Vitesse + maintenance + scalabilité contrôlée

MinIO (S3)
Documents (PV, reçus, médical, exports)

Redis + BullMQ
Queue jobs (worker NestJS)

Traefik
HTTPS + routing (www/app/api)

1. Résumé exécutif	vision & choix
2. Périmètre fonctionnel	modules
3. Architecture technique	topologie & domaines
4. Sécurité, audit, sauvegardes	MVP solide
5. Flux critique (paiement)	états & règles
6. Modèle de données minimal	MVP extensible
7. Écrans MVP	fédération/club/national/public

1) Résumé exécutif

Objectif

Centraliser, sécuriser et professionnaliser la gestion fédérale (sportif, administratif, médical, financier), avec une montée à 50 clubs et ~5000 utilisateurs, et des pics saisonniers (renouvellements de licences).

Choix d'architecture

- **Backend monolithe modulaire** (domaines isolés, contrats clairs).
- **API REST** documentée OpenAPI/Swagger (NestJS).
- **Async** via Redis + BullMQ (worker NestJS).

Garanties "pro" (MVP solide)

- **RBAC + scopes** (org/saison/compétition).
- **Audit trail** sur entités sensibles.
- **Backups testés** (PITR Postgres + réplication MinIO).

Améliorations "niveau pro"

- **Observabilité** (logs/metrics/alerting) + runbooks.
- **MFA obligatoire** pour admin/finance/médical.
- **Scan antivirus** async des documents sensibles.

Gouvernance data

- Contraintes DB (FK/unique/check) + index "listes chaudes".
- Référentiels de statuts/motifs (paiement, licence, médical).
- Rétention/purge par type (notamment médical).

2) Périmètre fonctionnel (modules)

IAM

RBAC + scopes | MFA optionnelle

Comptes, rôles et permissions. Accès fédération vs organisation (club/équipe nationale), saison, compétition.

Règle : *adhérent = joueur non affecté à un club* (pas d'affectation active).

Licences & Membres

Workflow saison

Création/renouvellement/archivage, catégories (âge/sexe/discipline), historique, paiement & activation.

Clubs & Organisations

pending / active

Base clubs, staff, documents officiels, validation administrative, communication FRMHG ↔ clubs.

Sportif

Calendrier | Matchs | Stats

Compétitions, équipes, matchs, feuille de match, événements, classements, convocations/stages.

Finance

Virement + reçu

Cotisations, facturation, subventions/sponsors, dashboards, export comptable.

Médical

Accès restreint

Visites, tests, résultats, rapports PDF, blessures. Audit renforcé.

Communication

Emails | Notifications | Messagerie

Actualités fédérales, messages internes, notifications workflow (paiement, licence, convocation).

Site web public (hors plateforme)

Vitrine FRMHG | Sous-domaine www

Présentation, actualités, calendrier/résultats en lecture, contact. Possibilité d'hébergement sur le même serveur via le reverse-proxy.

3) Architecture technique (1 serveur on-prem, Docker Compose)

Un seul serveur (pas de haute disponibilité). On compense avec **sauvegardes testées** et **plan de reprise**.

3.1 Topologie de déploiement

3.3 Alternative – Scénario Haute Disponibilité (HA)

Option progressive pour réduire le risque “1 serveur = panne totale” : on sépare **APP** et **DATA**, on met des **réplicas** côté application, et on introduit la réplication DB/stockage selon budget.

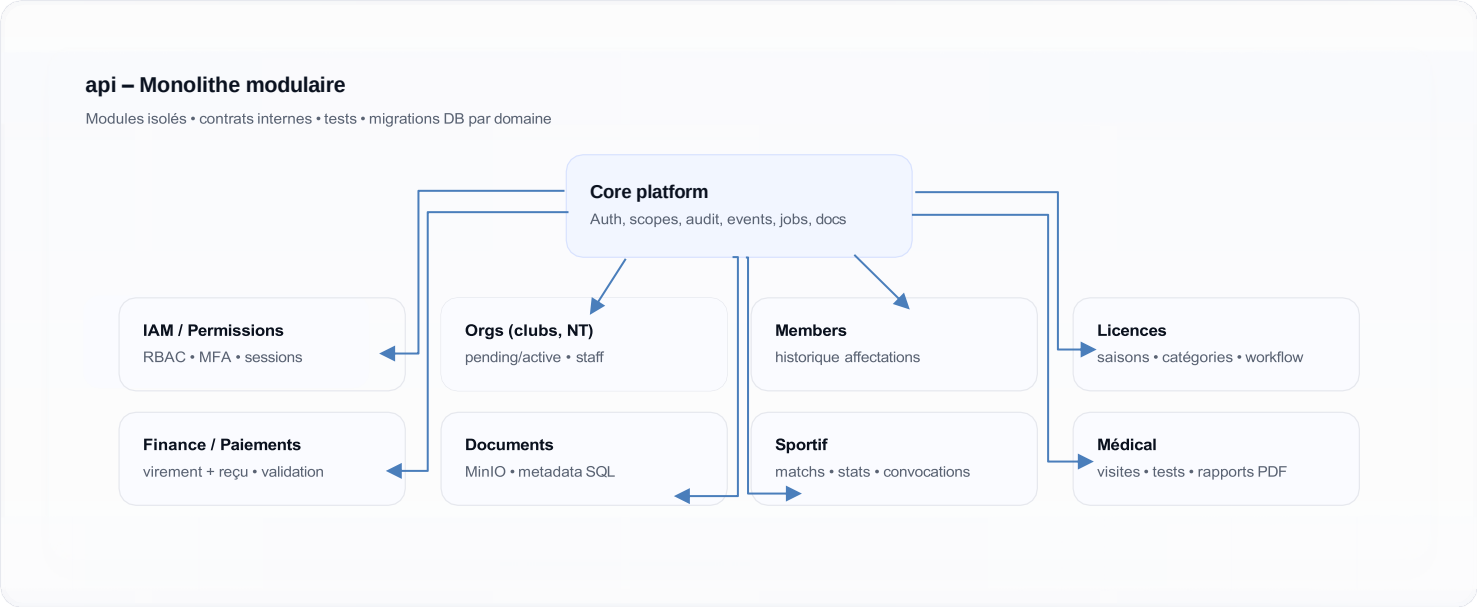
Sous-domaines

Recommandation : `www` → site public, `app` → plateforme, `api` → API (reverse-proxy unique).

Note

Même en HA, les **backups** restent obligatoires (HA ≠ backup). Côté applicatif : jobs idempotents + retries (BullMQ).

3.2 Backend monolithe modulaire (domaines)



Principe clé

Un seul backend, mais des **domaines strictement séparés** (dossiers, services, schémas de validation, politiques d'accès). On scale via **réplicas API** (si besoin) et surtout via **jobs async**.

4) Sécurité, audit, sauvegardes

IAM & accès

- **RBAC + scope** (fédération / org, saison, compétition).
- **MFA** recommandé (admin, finance, médical) — idéalement **obligatoire** pour ces profils.
- Sessions sécurisées (cookies HTTPOnly, rotation, rate-limit).

Audit trail (obligatoire)

- Paiements: validation/rejet, motif, décideur.
- Licences: activation/archivage, événements.
- Médical: créations/consultations, rapports.
- Droits/roles: changements et portée.

Protection des données

- HTTPS partout, secrets via variables/coffre.
- Chiffrement disque serveur.
- Rétention par type (médical, paiements, documents).

Sauvegardes & reprise (1 serveur)

- Postgres: dumps + **WAL/PITR** vers support externe.
- MinIO: versioning + sync/réplication externe.
- Tests de restauration réguliers (mensuel/trimestriel).

Observabilité & exploitation

Monitoring & alerting

- CPU/RAM/disque + alertes (saturation, volumes).
- Latence API, access logs proxy.
- Taille queue Redis, durée jobs worker.
- Santé Postgres/MinIO (read/write).

Runbooks & déploiements

- Procédures: restauration, incident sécurité, panne stockage.
- Versions figées (images taggées), fenêtre maintenance.
- Plan rollback documenté.

Risques majeurs & mitigation

Risque	Impact	Mitigation
Panne serveur (unique)	Indisponibilité totale	Backups externes + restauration testée, monitoring, procédure reprise
Perte/corruption DB	Données critiques	PITR (WAL), tests restore, accès DB restreints, contrôles intégrité
Perte documents (MinIO)	Reçus/rapports indisponibles	Versioning + sync externe, checksum, politiques d'accès
Compte sensible compromis	Fraude / fuite	MFA obligatoire, RBAC/scope strict, audit + alertes
Upload malveillant	Infection / exfiltration	Scan antivirus async, restrictions MIME/taille, stockage isolé
Pic annuel licences	Timeouts / lenteurs	Worker/queues, jobs idempotents, index DB, cache référentiels
Erreur workflow paiement	Activation à tort	Statuts stricts, validation fédérale, motifs structurés, double-contrôle optionnel
Données médicales	Conformité / réputation	Least privilege, masquage, audit consultations, rétention/purge

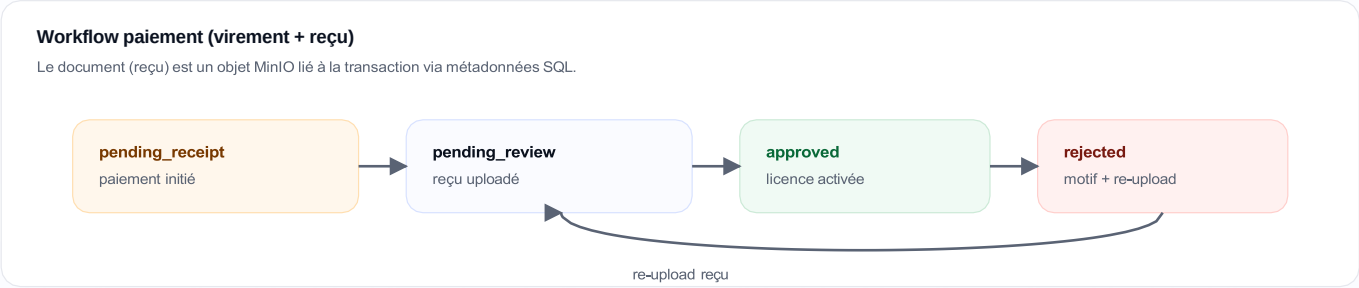
5) Flux critique – Paiement virement + upload reçu

États recommandés

- `pending_receipt` : paiement initié, reçu non fourni
- `pending_review` : reçu uploadé, attente validation
- `approved` : validé (déclenche activation licence, attestation)
- `rejected` : rejeté (motif + re-upload)

Règles

- Reçu stocké dans **MinIO** et lié à `payment_id` .
- Validation par la **fédération** (ou rôle finance).
- Audit complet: qui/quoi/quand + commentaire.



1) Modèle de données minimal (MVP extensible)

La base SQL conserve les entités métier et les métadonnées documents; les fichiers sont dans MinIO.

Domaine	Tables	Notes
IAM	`users` , `roles` , `user_roles`	Scopes: `scope_org_id` , `scope_season_id`
Orgs	`orgs` , `org_staff`	`orgs.type = club | national_team`
Membres	`members` , `member_org_history`	Adhérent = pas d'affectation club active
Licences	`seasons` , `categories` , `licenses` , `license_events`	Historique par saison + événements
Paiements	`payments` , `documents` (+ `payment_receipts`)	Reçus en MinIO; validation fédérale; audit
Médical	`medical_visits` , `injuries`	Rapport PDF via worker → MinIO
Sportif	`competitions` , `teams` , , `match_sheet` , `match_events`	Recalcul stats/classements via worker
Communication & Audit	`messages` , `notifications` , `audit_log`	Traçabilité "qui a fait quoi"

2) Écrans MVP (synthèse)

Fédération

- Dashboard / KPI
- Validation clubs (`pending → active`) Saisons, catégories, licences
- Paiements (file validation + reçus) Médical (paramétrage, visites,
- rapports)
- Sportif (compétitions, matchs, classements)
- Admin (IAM, rôles, audit)
- Gestion des stocks

Club

- Dashboard club
- Effectif & staff
- Licences (création/renouvellement)
- Paiement (upload reçu)
- Documents club
- Sportif (matchs, convocations)
- Messagerie

Équipe nationale

- Dashboard sélection
- Listes / convocations / stages
- Présences & planning
- Sportif (matchs/événements)
- (selon droits) médical sélectionnés

Public (site web)

- Présentation FRMHG
- Actualités/communiqués
- Calendrier & résultats (lecture)
- Contact (anti-spam)

Note

Le site public peut rester **statique** (coût minimal) ou consommer l'API en lecture (résultats/actualités), tout en gardant la plateforme isolée par sous-domaine et politiques de sécurité.

Principe
On prépare la scalabilité sans complexifier le MVP : séparation claire, jobs async, stockage objet, et discipline d'exploitation (observabilité + backups).

Stack utilisé :

- Reverse-proxy: Traefik
- Frontend: Next.js (TypeScript)
- public-web (site public www)
- web (plateforme app)
- UI: Tailwind CSS + shadcn/ui
- Backend: NestJS (Node.js + TypeScript) (api) + OpenAPI/Swagger
- Jobs async: NestJS worker + BullMQ
- Base de données: PostgreSQL
- Cache/Queue: Redis
- Stockage documents: MinIO (S3)